CAV-Bench: Can Code Agents Complete Long-Horizon C Program Verification? Supplementary Material

Anonymous submission

Corrections to HumanEval-X C++ Implementations

We use the original HumanEval descriptions as the authority for task semantics and cross-check the expected behavior against the canonical Python solutions (Chen et al. 2021). During formal verification, we found seven HumanEval-X C++ reference implementations whose behavior differed from the original semantics (Zheng et al. 2023). For each case, we constructed a concrete counterexample, corrected the C implementation to follow HumanEval, and then verified the corrected program. Table 1 summarizes the discrepancies and corrections. These discrepancies were not exercised by the released HumanEval-X tests.

These cases illustrate why passing the source benchmark tests was insufficient for constructing verified ground truth. Formalization required an explicit semantic comparison with the original task descriptions, and verification exposed behaviors outside the finite test set. We retain the corrected C implementations in CAV-Bench and record the changes so that they are not mistaken for semantics-preserving C++-to-C adaptations.

Reproducibility and Agent Setup

This section describes the files, agent commands, time budget, and instructions used in our experiments. It also explains how we decide whether a submitted artifact succeeds.

Agent Invocation and Runtime

All agents are launched by the same experiment runner. For each task, it creates a fresh workspace, provides the same instructions, records the run and its cost, and audits the submitted artifact.

Task information. The agent receives the target function, the task inputs, the QCP repository, workspace paths, and shared verification instructions. It receives no task-specific auxiliary annotation, proof outline, proof steps, or reference solution. Any task-specific formal definitions are part of the benchmark input. The instructions state which files may be read or changed, which documentation and examples may be used, how to run VCG and Rocq, and how success is checked.

Budget and outcomes. Each primary-task and proof-only run has a 90-minute wall-clock budget. The runner starts the harness once and does not automatically send another prompt, although the harness may perform its own iterations. Eleven interrupted runs were restarted in the same workspace without extending their total budget. When a run ends, the runner audits its submitted artifact. For the primary task, an audited submission is *Correct* only if its recorded end-to-end wall-clock time is at most 5,400 seconds. Audited completions exceeding that limit count as *Timeout*. Runs that exhaust the budget are also *Timeout*, and other submissions are *Wrong*. The agent's final message does not determine the result.

Agent commands. Codex, Claude Code, OpenCode, and Kimi Code receive the same task instructions and are checked by the same audit. The exact commands are shown below.

```
[Codex]
codex exec --json --skip-git-repo-check \
  --sandbox workspace-write --ephemeral \
  -C <qcp_root> -o <last_message_path> \
  --model <model> \
  --reasoning-effort <effort> \
  -

[Claude Code]
claude --print --bare --no-session-persistence \
  --permission-mode bypassPermissions \
  --add-dir <repo_root> <qcp_root> \
  --output-format stream-json --verbose \
  --model <model>

[OpenCode]
opencode run --format json --agent build \
  --model <model> \
  <prompt>

[Kimi Code]
kimi --print --work-dir <qcp_root> \
  --input-format text \
  --output-format stream-json \
  --model <model> --thinking
```

Checking Submitted Artifacts

Deterministic Audit

The deterministic audit checks the submitted artifact and decides success rather than relying on the agent's final message. As in the main paper, it reruns VCG and Rocq, checks that the fixed contract and executable implementation re-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Table 1: Corrections to HumanEval-X C++ reference implementations.

Task	Required behavior	Counterexample and original behavior	Correction
`p006`	Produce one maximum nesting depth for each nonempty space-delimited group.	On `"(()())"`, the C++ implementation returns `{1,2}` instead of `{2}` because it emits a result whenever the nesting depth returns to zero.	Treat spaces, rather than returns to depth zero, as group boundaries and compute one maximum depth per group.
`p009`	Compute the rolling maximum for an arbitrary integer sequence.	On `{-5,-3,-4}`, the C++ implementation returns `{0,0,0}` instead of `{-5,-3,-3}` because its running maximum starts at zero.	Initialize the running maximum from the first element (or an equivalent lower bound) while handling the empty input separately.
`p080`	Every three consecutive characters must be pairwise distinct.	The C++ implementation returns true on `"aab"`: it compares the third character with the first two but never compares the first two characters with each other.	Check all three pairs in every length-three window, including the first pair of the string.
`p086`	Sort characters within each word while preserving all spaces.	On an input consisting of two leading spaces followed by `"ba"`, the C++ implementation returns `"ab"` and drops both spaces.	Preserve every input space, including leading spaces and empty word boundaries, while sorting only the non-space characters of each word.
`p132`	Return true when the input contains a nested bracket subsequence of the form `[[]]`.	The string `"[][]]"` contains such a subsequence at positions 0, 2, 3, 4, but the C++ depth-based algorithm returns false.	Replace the depth heuristic with a four-state subsequence recognizer for `[`, `[`, `]`, `]`.
`p134`	Return true only when the final space-delimited word is a single alphabetic character.	The C++ implementation returns true on `"abc.e"`, even though the final `e` belongs to the same word, because it only checks that the preceding character is not a letter.	Require the final letter either to be the only character or to be immediately preceded by a space.
`p160`	Evaluate the constructed expression using the operator semantics of the original Python expression, including right-associative exponentiation.	For operands `{2,3,2}` and operators `{"**","**"}`, the C++ implementation evaluates $(2^3)^2 = 64$ instead of $2^{(3^2)} = 512$.	Evaluate exponentiation from right to left while retaining the specified precedence and left associativity of the other operators.

```
Task:
Verify the fixed C implementation against its contract. Complete
Auxiliary Annotation and Proof, and submit an artifact that passes
the audit. Do not change the contract or executable implementation.

Available tools and references:
- Use VCG to generate proof goals and Rocq to check proofs.
- You may read the supplied documentation, examples, task definitions,
  and Rocq libraries.
- You may import the supplied Rocq libraries in the proof.

Allowed changes:
- Change only auxiliary annotations and the proof.
- Do not change the fixed contract, executable C implementation,
  generated proof goals, or automatically generated proof files.
- Do not read reference answers, runner files, other task workspaces,
  or other disallowed material.

Verification loop:
1. Add or repair auxiliary annotations and run VCG.
2. Complete the proof and run Rocq.
3. If the proof needs a missing fact, add it to the auxiliary
   annotations, rerun VCG, and repair the proof.
4. Continue until the audit passes or the time budget is exhausted.
5. Do not use unfinished proofs, new axioms, or imports that bypass
   proof checking.

Audit and exit:
- Run <audit_script> after each complete attempt.
- Report Success only after the audit passes.
- An ordinary VCG, Rocq, or audit failure is not a reason to stop.
```

Figure 1: Core verification instructions supplied to every primary-task agent.

main unchanged, detects ways of avoiding the required proof, and verifies that the agent did not read disallowed material. The agent may run the same checks while working, but the experiment runner performs the final audit after the run ends.

Check	Acceptance condition
VCG and Rocq	VCG is rerun from the submitted auxiliary annotations and must regenerate every required proof goal. Rocq then checks the submitted proof against the regenerated goals and required task definitions.
Fixed inputs	The fixed contract must be unchanged. After comments and auxiliary annotations are removed from the submitted C file, its executable implementation must match the original input.
Proof integrity	The proof must exist and contain no unfinished proof, added axiom, forbidden import, edited generated goal, deleted obligation, or other way of avoiding the required proof.
Disallowed material	The run logs are checked to confirm that the agent did not read reference answers, other task workspaces, runner output, or other disallowed material.

The submitted artifact is accepted only if every check succeeds. This connects the checked proof to the fixed contract and implementation while preventing success through task modification or an invalid shortcut.

Contract Evaluation

A generated contract must do more than compile. It should accept the intended inputs, rule out unsafe C executions, and describe the required result on every relevant path. A contract can still be too weak, too restrictive, or tied too closely to one implementation even when the implementation can be proved against it.

Existing benchmarks check generated contracts through equivalence proofs, positive and negative examples, or faulty program variants (Thakur et al. 2025; Ye et al. 2026; Le-Cong, Le, and Murray 2025; Yang et al. 2026). These methods are useful, but no single method is sufficient here. QCP contracts describe both functional behavior and C memory. Tests cannot directly check every memory requirement or Rocq definition, while proving equivalence to a reference contract would add another difficult proof task. Checking only that the implementation satisfies the generated contract would also accept contracts that say too little.

For the additional end-to-end task, we therefore use an LLM judge as an initial contract-quality check. It receives the task description, fixed C implementation, generated contract, and any required Rocq definitions. It checks whether the contract covers the intended inputs, C safety, outputs, and program paths. This judge is not used in the primary task, which provides a fixed human-reviewed contract. The main part of the judge prompt is shown below.

The eight checks separate contracts that say too little from contracts that make an incorrect claim. A failed check rejects the contract, and insufficient evidence produces *Inconclusive*. Because this review is not a formal proof, it does not determine the primary fixed-contract score. After a generated contract passes this review, Auxiliary Annotation and Proof are still checked by VCG, Rocq, and the deterministic audit.

Table 2: Examples of generated contracts rejected by the contract review.

Task	Detected contract error
`p002` truncate	The postcondition only placed the return value in a range; it did not require the fractional part.
`p035` maximum	The postcondition required an input element but did not require the maximum, and it missed one return path.
`p059` prime factor	The precondition excluded legal prime inputs and imposed an unnecessarily small upper bound.
`p062` derivative	The contract did not connect the mathematical result to the returned array and allowed an empty result on ordinary inputs.
`p071` triangle area	The result condition reduced to `True` and therefore placed no requirement on the return value.

The primary task includes both Auxiliary Annotation and Proof because the two stages depend on each other. A failed proof may reveal a fact that must first be added to the auxiliary annotations. The agent must then rerun VCG and repair the proof. The additional proof-only task isolates Proof, while the primary task measures whether the agent can complete this coupled process.

```
Evaluate whether the candidate contract fully characterizes the target behavior on the task's legal input domain.

For each item, return Pass, Fail, or Inconclusive with one sentence of evidence from the contract, program, or
     Rocq definitions:
1. Legal inputs: does the contract exclude any intended input?
2. C safety: does it rule out runtime errors and provide the required memory access?
3. Required result: does it state the result rather than only shape, bounds, or an empty condition?
4. Correctness: is every claimed behavior consistent with the code?
5. Inputs and outputs: does it connect every relevant input and output?
6. Memory effects: does it fully describe every required change to memory?
7. Program paths: does it cover every observable branch?
8. Full behavior: does it determine the intended result for every legal input?

Treat explicit task constraints as the intended domain. Permit only additional restrictions needed for memory and
     integer safety. Reject undefined conditions, conditions equivalent to True, and definitions that merely
     repeat the implementation. Return Correct only if all eight items Pass; otherwise return Buggy or
     Inconclusive.
```

Figure 2: Prompt used to review generated contracts in the end-to-end task.

References

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Le-Cong, T.; Le, B.; and Murray, T. 2025. Can LLMs Reason About Program Semantics? A Comprehensive Evaluation of LLMs on Formal Specification Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21991–22014. Vienna, Austria: Association for Computational Linguistics.

Thakur, A.; Lee, J.; Tsoukalas, G.; Sistla, M.; Zhao, M.; Zetzsche, S.; Durrett, G.; Yue, Y.; and Chaudhuri, S. 2025. CLEVER: A Curated Benchmark for Formally Verified Code Generation. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Yang, F.; Li, X.; Wang, S.; An, J.; Sun, Z.; Feng, S.; Wang, W.; Wang, W.; Zhan, N.; and Xu, F. 2026. How Powerful Are LLMs in Generating Formal Program Specifications? In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*.

Ye, Z.; Yan, Z.; He, J.; Kasriel, T.; Yang, K.; and Song, D. 2026. VERINA: Benchmarking Verifiable Code Generation. In *The Fourteenth International Conference on Learning Representations*.

Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Wang, Z.; Shen, L.; Wang, A.; Li, Y.; Su, T.; Yang, Z.; and Tang, J. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5673–5684.